

TP 5 : Apprentissage supervisé et méthode des K plus proches voisins

Lola Etiévant

2025-2026

TP 5 : Apprentissage supervisé et méthode des K plus proches voisins

0. Chargement et Préparation des Données

0.1. Chargement du jeu de données

```
# Chargement des données
qualitative_vars = read.csv("../TP2/titanic_pre_processed_qualitative_vars.csv")
quantitative_vars = read.csv("../TP2/titanic_pre_processed_quantitative_vars.csv")
# Load the target and immediately convert to a factor vector
target_df = read.csv("../TP2/titanic_pre_processed_target.csv", stringsAsFactors = FALSE)
# On transforme immédiatement target en vecteur de facteurs (plus besoin de fonction helper)
target = as.factor(target_df$x)

# Suppression des observations avec 'Sex' manquant (moins de 0.2% des données)
isMissingSex = which(is.na(qualitative_vars$Sex) | qualitative_vars$Sex == "")
if (length(isMissingSex) > 0) {
  qualitative_vars = qualitative_vars[-isMissingSex, ]
  quantitative_vars = quantitative_vars[-isMissingSex, ]
  target = target[-isMissingSex]
}

# Conversion des variables explicatives en facteurs
qualitative_vars$Sex = as.factor(qualitative_vars$Sex)
qualitative_vars$Pclass = as.factor(qualitative_vars$Pclass)
qualitative_vars$withFam = as.factor(qualitative_vars$withFam)
qualitative_vars$Age_cat = as.factor(qualitative_vars$Age_cat)
```

1. Méthode des K plus proches voisins (KNN)

1.1. Rappels Théoriques

- **Hyperparamètre** : L'hyperparamètre est , le nombre de voisins. On le choisit par validation croisée ou via un ensemble de validation pour équilibrer le biais et la variance.
- **Partage des données** : On divise le jeu en 3 parties : **Train** (apprentissage), **Validation** (sélection de), et **Test** (évaluation finale des performances).

1.2. Codage binaire (dummy coding) et encoding

```
library(caret)

# Transformation des variables qualitatives en variables muettes (0/1)
make_dummies = function(var_name, data) {
  encoder = dummyVars(paste0("~", var_name), data = data)
  as.data.frame(predict(encoder, newdata = data))
}

qualitative_vars_dummy = data.frame(
  make_dummies("Sex", qualitative_vars),
  make_dummies("withFam", qualitative_vars)
)

# Transformation de Pclass en numérique pour le calcul des distances KNN
qualitative_vars_encoded = data.frame(Pclass = as.numeric(as.character(qualitative_vars$Pclass)))
```

1.3. Variables explicatives finales

```
# Assemblage du dataframe de prédiction
predictive_vars = data.frame(qualitative_vars_encoded, qualitative_vars_dummy, quantitative_vars)

# Nettoyage des colonnes redondantes pour éviter la colinéarité et simplifier le modèle
predictive_vars$Sex.male = NULL
predictive_vars$withFam.0 = NULL
predictive_vars$SibSp = NULL
predictive_vars$Parch = NULL

# Pourquoi ne pas normaliser tout de suite ?
# Réponse : Pour éviter le Data Leakage (fuite d'information du Test vers le Train).
```

2. KNN avec partage Holdout (80/10/10)

2.1. Partage du jeu de données

```
set.seed(1234)
n = nrow(predictive_vars)

# Extraction du Test (10%)
inTest = sample(1:n, size = round(0.1 * n))
notinTest = (1:n)[-inTest]

# Extraction de la Validation (10%) parmi le reste
inValidation = sample(notinTest, size = round(0.1 * n))
```

```
# Création des ensembles (les targets sont des vecteurs directs)
predictive_vars_test      = predictive_vars[inTest,]
target_test              = target[inTest]

predictive_vars_validation = predictive_vars[inValidation,]
target_validation        = target[inValidation]

predictive_vars_train      = predictive_vars[-c(inTest, inValidation),]
target_train              = target[-c(inTest, inValidation)]
```

2.2. Pre-processing (Imputation et Normalisation)

```
# On définit la transformation sur le Train UNIQUEMENT
Proc = preProcess(predictive_vars_train, method = c("medianImpute", "range"))

# On applique la transformation aux trois ensembles
predictive_vars_train      = predict(Proc, predictive_vars_train)
predictive_vars_validation = predict(Proc, predictive_vars_validation)
predictive_vars_test       = predict(Proc, predictive_vars_test)
```

2.3. Sélection de l'hyperparamètre

```
library(class)
KGrid = 1:20
accuracy_val = recall_val = precision_val = NULL

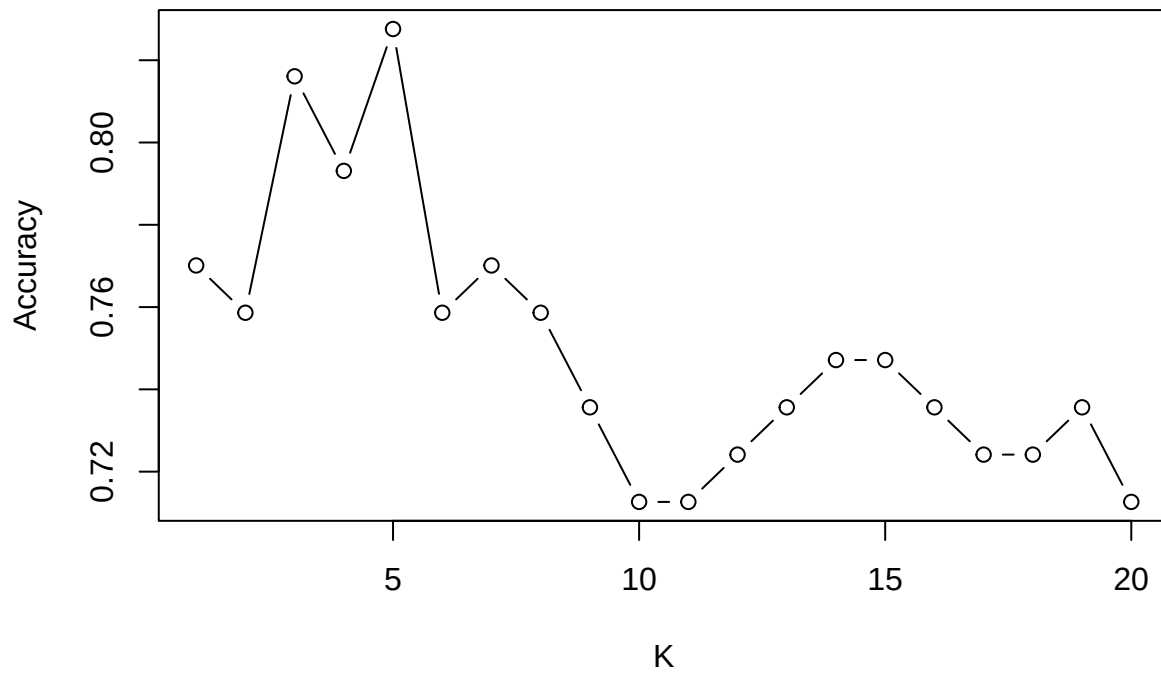
for(K in KGrid){
  set.seed(1234)
  # Application du KNN
  target_pred = knn(train = predictive_vars_train,
                    test  = predictive_vars_validation,
                    cl     = target_train, k = K)

  # Calcul de la matrice de confusion
  cm = confusionMatrix(data = target_pred, reference = target_validation, positive = "Survivant")

  accuracy_val = c(accuracy_val, cm$overall["Accuracy"])
  recall_val   = c(recall_val, cm$byClass["Sensitivity"])
  precision_val = c(precision_val, cm$byClass["Pos Pred Value"])
}

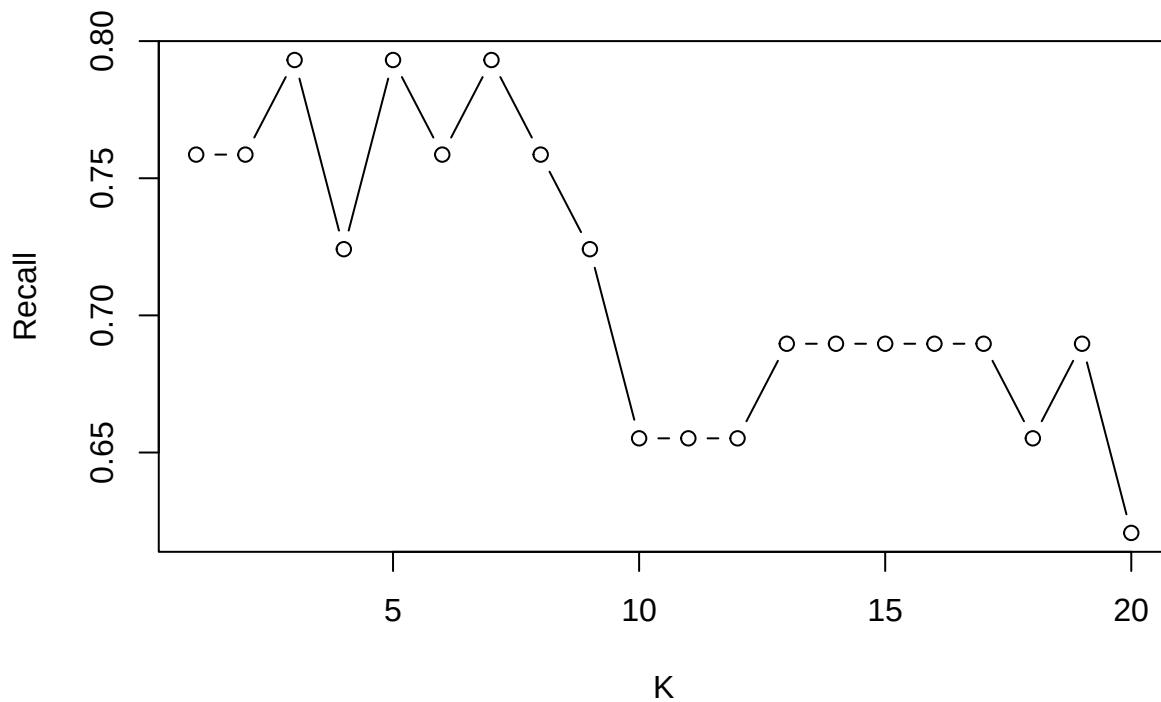
# Visualisation
plot(KGrid, accuracy_val, type = 'b', main="Sélection de K (Validation)", xlab="K", ylab="Accuracy")
```

Sélection de K (Validation)



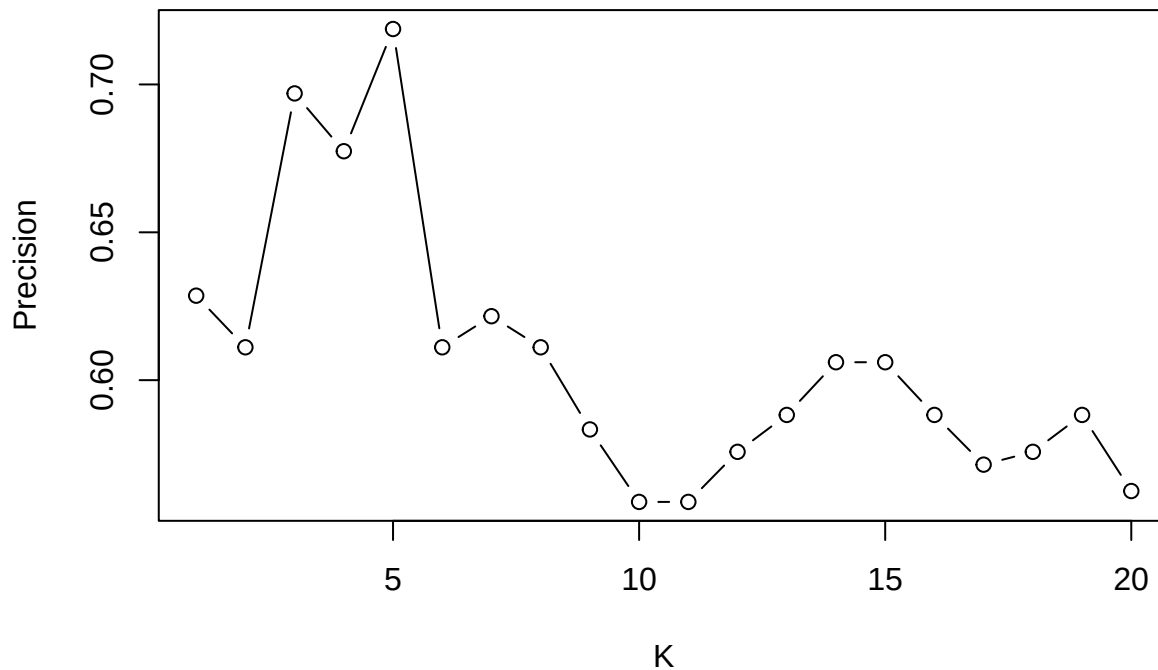
```
plot(KGrid, recall_val, type = 'b', main="Sélection de K (Validation)", xlab="K", ylab="Recall")
```

Sélection de K (Validation)



```
plot(KGrid, precision_val, type = 'b', main="Sélection de K (Validation)", xlab="K", ylab="Precision")
```

Sélection de K (Validation)



3. KNN avec Cross-Validation (CV)

3.1. Procédure de Cross-Validation

```
# On utilise tout ce qui n'est pas dans le Test final
predictive_vars_remain = predictive_vars[-inTest,]
target_remain          = target[-inTest]

n_remain = nrow(predictive_vars_remain)
nfolds   = 10
folds     = sample(1:nfolds, n_remain, replace = TRUE)

accuracy_fold = NULL

for(k in 1:nfolds){
  inFold = which(folds == k)

  # Split interne
  train_cv = predictive_vars_remain[-inFold,]
  val_cv   = predictive_vars_remain[inFold,]
  t_train  = target_remain[-inFold]
  t_val    = target_remain[inFold]

  # Pre-processing interne pour éviter le leakage
  Proc_cv  = preProcess(train_cv, method = c("medianImpute", "range"))
```

```

train_cv = predict(Proc_cv, train_cv)
val_cv   = predict(Proc_cv, val_cv)

acc_K = NULL
for(K in KGrid){
  pred = knn(train = train_cv, test = val_cv, cl = t_train, k = K)
  acc_K = c(acc_K, confusionMatrix(pred, t_val)$overall["Accuracy"])
}
accuracy_fold = rbind(accuracy_fold, acc_K)
}

# Calcul des moyennes par colonne (pour chaque K)
cv_accuracies = colMeans(accuracy_fold)
best_k_cv     = KGrid[which.max(cv_accuracies)]

```

3.2. Performance finale sur le jeu de Test

```

# 1. Re-calcul du preprocessing sur l'ensemble 'remain' (Train + Val)
Proc_final = preProcess(predictive_vars_remain, method = c("medianImpute", "range"))

train_final_scaled = predict(Proc_final, predictive_vars_remain)
test_final_scaled  = predict(Proc_final, predictive_vars_test)

# 2. Apprentissage final avec le meilleur K trouvé par CV
final_predictions = knn(train = train_final_scaled,
                        test  = test_final_scaled,
                        cl    = target_remain,
                        k     = best_k_cv)

# 3. Métriques finales
cm_final = confusionMatrix(final_predictions, target_test, positive = "Survivant")
print(cm_final)

```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction  Décédé Survivant
##   Décédé      34      4
##   Survivant   14     35
##
##           Accuracy : 0.7931
##           95% CI : (0.6929, 0.8725)
##   No Information Rate : 0.5517
##   P-Value [Acc > NIR] : 2.24e-06
##
##           Kappa : 0.5915
##
##   Mcnemar's Test P-Value : 0.03389
##
##           Sensitivity : 0.8974
##           Specificity : 0.7083

```

```
##          Pos Pred Value : 0.7143
##          Neg Pred Value : 0.8947
##          Prevalence : 0.4483
##          Detection Rate : 0.4023
##    Detection Prevalence : 0.5632
##    Balanced Accuracy : 0.8029
##
##    'Positive' Class : Survivant
##
```